
NEURA CPP API

Release 1.1

Neura Robotics GmbH

Jul 11, 2025

CONTENTS

1	Introduction to Neura C++ API	1
2	Setting up Neura C++ API	2
2.1	Installing Neura C++ API	2
2.1.1	For Ubuntu	2
2.2	Setting up remote connection with Robot's Software	2
3	Features of Neura C++ API	4
3.1	Functionalities	4
3.2	Neura C++ Client	5
3.2.1	Robot(unsigned int robot_num = 0)	5
3.2.2	powerON()	6
3.2.3	powerOFF()	6
3.2.4	getLastMachineState()	6
3.2.5	getLastJointStates()	7
3.2.6	getLastCartesianState()	7
3.2.7	servoJ()	8
3.2.8	speedJ()	8
3.2.9	servoX()	9
3.2.10	speedX()	9
3.2.11	moveLinearOnline()	10
3.2.12	stopMoveLinearOnline()	10
3.2.13	getForwardKinematics()	10
3.2.14	getInverseKinematics()	11
3.2.15	getCurrentJointLimits()	11
3.2.16	getCurrentCartesianLimits()	11
3.2.17	getExternalWrench()	12
3.2.18	getTrajectoryScalingFactor()	12
3.2.19	getErrorCode()	12
4	Examples	13
4.1	Building an example	13
4.2	Example Usage	13
4.2.1	Robot Object Initialization	14
4.2.2	Reading Machine State	15
4.2.3	Reading Joint States	16
4.2.4	Reading Cartesian States	17
4.2.5	SpeedJ	18
4.2.6	ServoJ	20
4.2.7	ServoX	23

4.2.8	Move Linear Online	26
4.2.9	Joint Limits	29
4.2.10	Cartesian Limits	30
4.2.11	Forward Kinematics	31
4.2.12	Inverse Kinematics	31
4.2.13	SpeedX	32
4.2.14	Reading Force Torque Sensor States	34
4.2.15	Trajectory Scaling Factor	36
4.2.16	Error Codes	38
4.3	Guide for integrating API with custom projects	40
4.3.1	Required Configuration Steps	40
4.3.2	Sample <i>CMakeLists.txt</i>	41
5	Robot Safety Configurations	42
5.1	Safety Configurations	42
5.2	Configuring and Switching Between Safety Modes	42
5.2.1	Lara	42
5.2.2	MAiRA	43
6	Changelog and Versioning Information	45
6.1	1.0 (13-11-2024)	45
6.2	1.1 (31-01-2025)	45

INTRODUCTION TO NEURA C++ API

The Neura C++ API is a versatile, extendable, and robot-agnostic interface for Neura's control software, made specifically for application developers. This comprehensive API enables robust integration and control of Neura's systems, allowing developers to leverage and customize the control functionalities to meet precise application requirements. Designed for flexibility and ease of use, it supports a wide variety of robotic configurations without necessitating extensive code modifications.

The API provides real-time control at a frequency of 1 kHz through four distinct command interfaces:

1. Joint position commands.
2. Joint velocity commands.
3. Cartesian pose commands.
4. Cartesian velocity commands.

Simultaneously, it provides access to 1 kHz measurements of:

- Joint Data: Real-time joint position, velocity and torque.
- Cartesian Data: Real-time cartesian position and velocity.
- Joint and Cartesian Limits: Limits on joint and cartesian values that are set in the GUI.
- Force Torque Sensor Data: Real-time forces and moments from the force torque sensor.
- Trajectory Scaling Factor: Scaling factor between 0 and 1 that is used to scale down trajectories when the robot is near any limits.
- Error Codes: Any error/warning code that could be thrown during the motion.

Additionally, the API allows control over the robot's power state, including the capability to power the robot on and off.

Furthermore, the API grants access to the robot model library, which encompasses:

- Forward Kinematics: Calculation of the end-effector position and orientation based on a specified set of robot joint states.
- Inverse Kinematics: Calculation of the joint configurations required to achieve a given cartesian pose for the end-effector.

SETTING UP NEURA C++ API

2.1 Installing Neura C++ API

2.1.1 For Ubuntu

1. Please download and unzip the installation package provided by Neura.

Note: If you did not receive the installation package with the robot, please contact your project manager.

2. Navigate to the unzipped folder and run the installation script named *installation_script.sh* using the following command:

```
. installation_script.sh
```

2.2 Setting up remote connection with Robot's Software

1. Begin by connecting the Ethernet cable to the robot's control box. The location of the LAN port may vary depending on the model — it could be accessible externally or may require opening the control box.
2. Setup the ethernet connection IP. The address should be in the same IP network of the robot.

Note: The IP address should be of the form – 192.168.2.x (x should be different than 13/14/15/20)

3. Change the host address to be different than 13/14/15/20, for example, 100. So the complete address would be 192.168.2.100.
4. Open a terminal and try to ping the robot's IP. If transmitting and receiving is successful, then the connection is successful.

```
ping 192.168.2.13
```

Cancel **Wired** Apply

Details Identity **IPv4** IPv6 Security

IPv4 Method

☐ Automatic (DHCP) ☐ Link-Local Only

☒ Manual ☐ Disable

Addresses

Address	Netmask	Gateway	
192.168.2.100	255.255.255.0	192.168.2.13	✕
			✕

DNS Automatic **ON**

Separate IP addresses with commas

Routes Automatic **ON**

Address	Netmask	Gateway	Metric	
				✕

Fig. 1: Connecting to the robot.

```

~$ ping 192.168.2.13
PING 192.168.2.13 (192.168.2.13) 56(84) bytes of data.
64 bytes from 192.168.2.13: icmp_seq=1 ttl=64 time=0.284 ms
64 bytes from 192.168.2.13: icmp_seq=2 ttl=64 time=0.243 ms
64 bytes from 192.168.2.13: icmp_seq=3 ttl=64 time=0.249 ms
64 bytes from 192.168.2.13: icmp_seq=4 ttl=64 time=0.170 ms
^C
--- 192.168.2.13 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3059ms
rtt min/avg/max/mdev = 0.170/0.236/0.284/0.044 ms

```

Fig. 2: Successful connection to the robot.

FEATURES OF NEURA C++ API

The Neura C++ API provides a Robot class, which grants access to all functionalities and static parameters of the robot. These capabilities are accessible through the methods of a Robot class instance.

3.1 Functionalities

S.No.	Functionality	Action
1.	<code>neura::Robot</code>	Robot class which allows access to all robot functionalities.
2.	<code>neura::Robot::Robot()</code>	Instantiate a robot class either with default serial number (0) or provide a serial number.
3.	<code>neura::Robot::powerON()</code>	Power On the robot when robot is in external mode.
4.	<code>neura::Robot::powerOFF()</code>	Power Off the robot when robot is in external mode.
5.	<code>neura::Robot::getLastMachineState()</code>	Get different status of the robot states and modes.
6.	<code>neura::Robot::getLastJointStates()</code>	Get the joint position, velocity and torque states from the robot.
7.	<code>neura::Robot::getLastCartesianState()</code>	Get the cartesian position, velocity and acceleration states of the tcp from the robot.
8.	<code>neura::Robot::servoJ()</code>	Perform joint position control of the robot in joint space by providing target positions, velocities and accelerations as arguments.
9.	<code>neura::Robot::speedJ()</code>	Perform velocity control of the robot in joint space by providing target velocities and accelerations as arguments.
10.	<code>neura::Robot::servoX()</code>	Perform cartesian position control of the robot's TCP in cartesian space by providing target positions, velocities and accelerations as arguments.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
11.	<code>neura::Robot::speedX()</code>	Perform velocity control of the robot in cartesian space by providing target velocities and accelerations as arguments.
12.	<code>neura::Robot::moveLinearOnline()</code>	Perform cartesian position control of the robot's TCP in cartesian space by providing target positions, velocities and accelerations as arguments. The function has inbuilt interpolator to interpolate between current position and target.
13.	<code>neura::Robot::stopMoveLinearOnline()</code>	Function to stop move linear online if motion is running.
14.	<code>neura::Robot::getCurrentJointLimits()</code>	Function to get the joint position and velocity limits set through GUI.
15.	<code>neura::Robot::getCurrentCartesianLimits()</code>	Function to get the cartesian position and velocity limits of end effector set through GUI.
16.	<code>neura::Robot::getForwardKinematics()</code>	Function to get forward kinematics of a specific set of joint angles.
18.	<code>neura::Robot::getExternalWrench()</code>	Function to get the external wrench coming from force torque sensor connected to flange of the robot.
17.	<code>neura::Robot::getInverseKinematics()</code>	Function to get inverse kinematics of a specific pose.
18.	<code>neura::Robot::getTrajectoryScalingFactor()</code>	Function to get trajectory scaling factor due to robot being near safety limits.
19.	<code>neura::Robot::getErrorCode()</code>	Function to get the error codes from the software.

3.2 Neura C++ Client

3.2.1 Robot(unsigned int robot_num = 0)

Constructs an interface object ready to communicate with a robot specified by its serial number. Multiple instances will use the same communication client in the background.

Function Signature:

Robot(unsigned int robot_num)

Parameters:

- *robot_num*: The serial number (from integer digits only) of the target robot.

3.2.2 powerON()

Function to Power On the robot. The function would only work when the robot is in external mode. External Mode can be triggered through GUI.

Function Signature:

powerON()

Returns:

- *true*: Power On command is successful.
- *false*: Power On command is unsuccessful.

3.2.3 powerOFF()

Function to Power Off the robot. The function would only work when the robot is in external mode. External Mode can be triggered through GUI. PowerOn must be called before using this function from the client, otherwise robot will not power off.

Function Signature:

powerOFF()

Returns:

- *true*: Power Off command is successful.
- *false*: Power Off command is unsuccessful.

3.2.4 getLastMachineState()

This function returns the last received machine state if available. Only if at least one value has been received since the instantiation of this class - the target will be overwritten with the last value, and the function will return true. The first 7 bits are described as follows:

- **Bit 0 (ROB ON)**: 0 = Simulation mode, 1 = Real mode.
- **Bit 1 (NO MOTION)**: 0 = Motion occurring, 1 = No motion.
- **Bit 2 (ZeroG ON)**: 0 = Zero gravity off, 1 = Zero gravity on.
- **Bit 3 (Singularity)**: 0 = No singularity, 1 = Singularity present.
- **Bit 4 (Ext/Int)**: 0 = Internal mode, 1 = External mode.
- **Bit 5 (Error)**: No error, 1 = Error occurred.
- **Bit 6 (Collision detected)**: 0 = No collision, 1 = Collision detected.
- **Bit 7 (REAL/SIM)**: 0 = Simulation, 1 = Real.
- **Bit 8 - 23**: Reserved.
- **Bit 24 - (Cart/Joint)**: 0 = Joint, 1 = Cartesian.
- **Bit 25 - (Pos/Torque)**: 0 = Position, 1 = Torque.
- **Bit 28 - 31 (DOF Information)**: Convert binary to decimal to get DOF value.

Function Signature:

getLastMachineState()

Parameters:

- *status_word* is a target uint64 variable. The first 7 bits of this status word represent various machine states, where each bit is either 0 or 1 depending on the specific condition of the machine.

Returns:

- *true*: Status word has been successfully retrieved.
- *false*: Buffer is empty.

3.2.5 getLastJointStates()

Reports the last received values for angle position, velocity and torque of each joint drive. This function returns the last received values for the angular position, velocity, and torque of each joint in the robot. The length of the vectors depends on the robot's degrees of freedom (DOF) (6 or 7).

Function Signature:

```
getLastJointStates(std::vector<double>& position, std::vector<double>& velocity,
std::vector<double>& torques, double& timestamp)
```

Parameters:

- *position* A vector containing the angular positions of each joint (in radians).
- *velocity* A vector containing the angular velocities of each joint (in radians per second).
- *torques* A vector containing the torques of each joint (in Newton-meter).
- *timestamp* A variable representing the unix time since the robot interface started publishing data.

Returns:

- *true*: if values have been successfully received since the function was called.
- *false*: if the client-side buffer is empty and no data has been received.

3.2.6 getLastCartesianState()

Retrieves the most recent Cartesian state of the Tool Center Point (TCP). This function returns the last received values for the position, orientation, translational velocity, and rotational velocity of the robot's Tool Center Point (TCP). The length of the vectors is typically 3 for position, translational velocity, and rotational velocity and 4 for quaternion orientation (qw, qx, qy, qz).

Function Signature:

```
getLastCartesianState(std::vector<double>& position, std::vector<double>& quaternion,
std::vector<double>& trans_vel, std::vector<double>& rot_vel, double& timestamp)
```

Parameters:

- *position* A vector containing the Cartesian position of the TCP (x, y, z in meters).
- *quaternion* A vector representing the orientation of the TCP in quaternion form (qw, qx, qy, qz).
- *trans_vel* A vector containing the translational velocity of the TCP (in meters per second).
- *rot_vel* A vector containing the rotational velocity of the TCP (omegax, omegay, omegaz in radians per second).
- *timestamp* A variable representing the unix time since the robot interface started publishing data.

Returns:

- *true*: if values have been successfully received since the function was called.

- *false*: if the client-side buffer is empty and no data has been received.

3.2.7 servoJ()

Controls the robot by specifying target joint positions, velocities, and accelerations. This function sets the target joint positions, velocities, and accelerations for the robot. It is suggested to use an interpolator to avoid sudden jumps in the joint movement and not give far away targets. None of the velocities must exceed the thresholds defined in the GUI. The interpolator can be used to change targets online, robot will adapt the trajectory while being within the constraints. The dimension of the position, velocity, and acceleration vectors corresponds to the robot's degrees of freedom (DOF)(6 or 7). This function should be called cyclically.

Function Signature:

servoJ(std::vector<double>& position, std::vector<double>& velocity, std::vector<double>& acceleration)

Parameters:

- *position* A vector containing the target angular positions for each joint (in radians).
- *velocity* A vector containing the target angular velocities for each joint (in radians per second).
- *acceleration* A vector containing the target angular accelerations for each joint (in radians per second squared). Max acceleration value is 5 radian per second squared.

Returns:

- *0*: if function call is successfully executed.
- *error code*: if any of the parameters exceed the allowed threshold or other errors occur.

3.2.8 speedJ()

Controls the robot by specifying joint velocities. This function sets the velocity of each robot joint. The velocities are ramped up smoothly to avoid sudden jumps. None of the velocities must exceed the thresholds defined in the GUI. The dimension of the velocity and acceleration vectors corresponds to the robot's degrees of freedom (DOF)(6 or 7). This function should be called cyclically.

Function Signature:

speedJ(std::vector<double>& velocity, std::vector<double>& acceleration)

Parameters:

- *velocity* A vector containing the target angular velocities for each joint (in radians per second).
- *acceleration* A vector containing the target angular accelerations for each joint (in radians per second squared). Max acceleration value is 5 radian per second squared.

Returns:

- *0*: if function call is successfully executed.
- *error code*: if any of the parameters values exceed the allowed threshold or other errors occur.

3.2.9 servoX()

Controls the robot by specifying target Cartesian positions, velocities, accelerations, and proportional gain. This function sets the target Cartesian positions, velocities, accelerations, and a proportional gain controller for the robot. It controls the robot's Tool Center Point (TCP) in Cartesian space. None of the velocities must exceed the thresholds defined in the GUI. It is suggested to use an interpolator to avoid sudden jumps in the robot's movement. The interpolator can be used to change targets online, robot will adapt the trajectory while being within the constraints. This function should be called cyclically.

Function Signature:

servoX(std::vector<double>& position, std::vector<double>& velocity, std::vector<double>& acceleration, double& proportional_gain)

Parameters:

- *position* A vector containing the target Cartesian positions of length 7 (translation (in m/s) and orientation) (x, y, z, qw, qx, qy, qz).
- *velocity* A vector containing the target Cartesian velocities of length 6 (translation (in m/s) and rotational (radian/s)) (x_dot, y_dot, z_dot, omega_x, omega_y, omega_z).
- *acceleration* A vector containing the target Cartesian acceleration of length 6 (translation (in m/s²) and rotational (radian/s²)) (x_ddot, y_ddot, z_ddot, alpha_x, alpha_y, alpha_z).
- *proportional_gain* A scalar representing the proportional gain for the control loop. Proportional gain is between [0.2, 100].

Returns:

- *0*: if function call is successfully executed.
- *error code*: if any of the parameters exceed the allowed threshold or other errors occur.

3.2.10 speedX()

Sends a cartesian velocity-only command and acceleration command to the robot

Function Signature:

speedX(std::vector<double>& velocity, std::vector<double>& acceleration)

Parameters:

- *velocity* A vector of target cartesian velocities, i.e. x_dot, y_dot, z_dot, omega_x, omega_y, omega_z
- *acceleration* A vector of target cartesian acceleration, i.e. x_dot_dot, y_dot_dot, z_dot_dot, alpha_x, alpha_y, alpha_z

Returns:

- *0*: if function call is successfully executed.
- *error code*: if any of the parameters exceed the allowed threshold or other errors occur.

3.2.11 moveLinearOnline()

Run Cartesian motion/ServoX with inbuilt interpolator to move the robot between current position and final target. No need for interpolator to be called on client side. The program should not terminate after calling this function, either sleep can be added or some other process can be run. Please see examples for more information. Function can be called multiple times to change target during the motion.

Function Signature:

```
moveLinearOnline(std::vector<double> &position, std::vector<double> &velocity,
std::vector<double> &acceleration)
```

Parameters:

- *position* A vector containing the target Cartesian positions (x, y, z in meters, qw, qx, qy, qz).
- *velocity* A vector containing the maximum Cartesian velocities that is to be reached during motion with x_dot, y_dot, z_dot, omega_x, omega_y, omega_z
- *acceleration* A vector containing the maximum Cartesian acceleration that is to be reached during motion with x_ddot, y_ddot, z_ddot, alpha_x, alpha_y, alpha_z

Returns:

- *0*: if function call is successfully executed.
- *error code*: if any of the parameters exceed the allowed threshold or other errors occur.

3.2.12 stopMoveLinearOnline()

Stops movelinearonline motion if its running.

Function Signature:

```
stopMoveLinearOnline()
```

Returns:

- *true*: Stop command is successful.
- *false*: Stop command is unsuccessful.

3.2.13 getForwardKinematics()

Get the Forward Kinematics pose of a set of joint angles

Function Signature:

```
getForwardKinematics(std::vector<double> & request, std::vector<double> & feedback)
```

Parameters:

- *request* Set of joint angles(in radians) with length equal to degree of freedom
- *feedback* Resulting pose sent through reference

Returns:

- *true*: forward kinematics call successful.
- *false*: forward kinematics call not successful.

3.2.14 getInverseKinematics()

Get the Inverse Kinematics (set of joint angles) for a specific pose.

Function Signature:

getInverseKinematics(std::vector<double>& request, std::vector<double>& feedback)

Parameters:

- *request* Requested pose (X,Y,Z,qw,qx,qy,qz) (translation in meters).
- *feedback* Resulting set of joint angles sent through reference.

Returns:

- *true*: inverse kinematics call successful.
- *false*: inverse kinematics call not successful.

3.2.15 getCurrentJointLimits()

Get the Joint Limits (position and velocity) of the robot set through GUI.

Function Signature:

getCurrentJointLimits(std::vector<double>& limits)

Parameters:

- *limits* The first elements are positive position limits for each joint (in radians), next elements are negative position limits for each joint (in radians), next elements are positive velocity limits for each joint (in radians/second) and last set of elements are negative velocity limits for each joint (in radians/second).

Returns:

- *true*: call successful.
- *false*: call not successful.

3.2.16 getCurrentCartesianLimits()

Get the Joint Limits (position and velocity) of the robot set through GUI.

Function Signature:

getCurrentCartesianLimits(std::vector<double>& limits)

Parameters:

- *limits* The first three elements are positive X,Y,Z position limits (in meters), next 3 elements are negative X,Y,Z position limits (in meters), next element is translation velocity (in meters/second), next element is rotational velocity (in radians/second).

Returns:

- *true*: call successful.
- *false*: call not successful.

3.2.17 getExternalWrench()

Get the external wrench (forces and moments) coming from the force torque sensor connected to flange of the robot.

Function Signature:

getExternalWrench(std::vector<double>& external_wrench)

Parameters:

- *external_wrench* The first three elements contain forces (in Newton), i.e. Fx, Fy, Fz, and the next three elements contain moments (in Newton-meters), i.e. Mx, My, Mz.

Returns:

- *true*: call successful.
- *false*: call not successful.

3.2.18 getTrajectoryScalingFactor()

Get the trajectory scaling factor providing a factor between 0 & 1 used to scale trajectories internally when the robot reaches near the safety limits. Safety limits include all the position and velocity limits set through GUI, robot safety configuration like cobot mode (max TCP velocity 250mm/s) , safeguard stop. More information can be found in robot safety configuration chapter [Robot Safety Configurations](#).

Function Signature:

getTrajectoryScalingFactor()

Returns:

- *double*: scaling factor between [0,1].

3.2.19 getErrorCode()

Get the error code from the robot software. Please refer error mapping to get the meaning behind the error.

Function Signature:

getErrorCode()

Returns:

- *int*: error code.

EXAMPLES

4.1 Building an example

To build an example, navigate to the examples directory of the unzipped directory and execute the following commands:

```
mkdir build
cd build
cmake ..
make -j5
```

These steps will create the executable files in the *build* directory. You can then run any of the available executables as required.

For instance, to run the “reading_machine_state” feature, use the corresponding executable, *reading_machine_state*, located in the *build* directory. Run it with the following command in the terminal while in the *build* directory:

```
./reading_machine_state
```

4.2 Example Usage

This section demonstrates how to use the *Robot* class.

Important: When using the Robot object in code, replace the placeholder value of `robot_serial_number` with your specific robot’s serial number. The serial number can be found on the robot base or in the GUI under the “Info” section. Figures 1 and 2 show where to locate the serial number on the robot and in the GUI. **Use only the numeric portion following the “NR” prefix.**



Fig. 1: The arrow indicates the location of the serial number on the robot base.

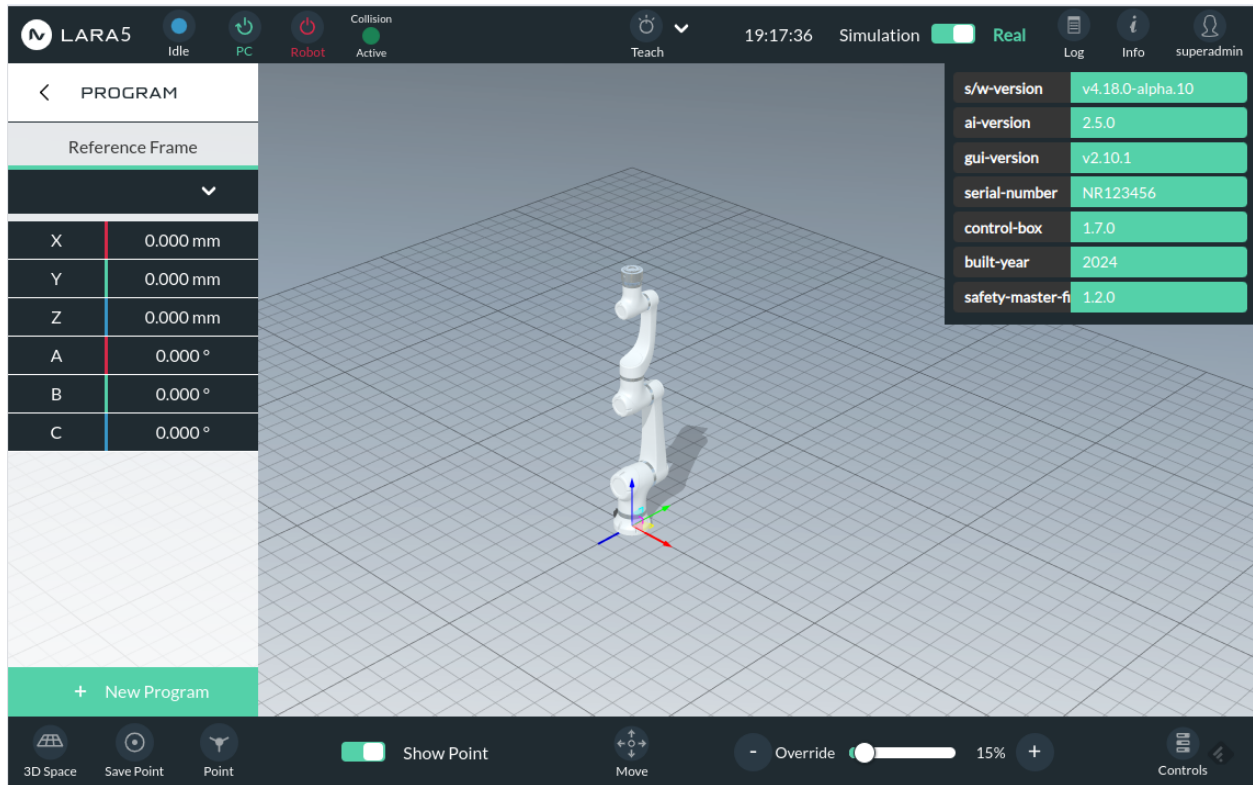


Fig. 2: Click the “Info” button at the top left of the GUI to view the robot’s serial number.

Important: Adjust the degree of freedom (dof) in each example to match your specific robot model. For most examples, set *dof* = 6 for Lara and *dof* = 7 for MaiRA. This adjustment is required for all examples except ServoX and Move Linear Online.

4.2.1 Robot Object Initialization

The following example demonstrates how to create a *Robot* object in code using the `neura::Robot::Robot()` function:

```

1  #include <iostream>
2  #include <neura_cpp_api/robot.hpp>
3
4  int main(int argc, char** argv)
5  {
6      size_t dof = 6;
7      unsigned robot_serial_number = 0; //Numerical number on serial number tag
8      auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
9      if (!p_robot)
10     {
11         throw std::runtime_error("Could not instantiate robot interface no. " +
12                                 std::to_string(robot_serial_number));

```

(continues on next page)

(continued from previous page)

```

13     return -1;
14 }
15     std::cout << "Created robot client no. " << std::to_string(robot_serial_number) << "\n";
16     return 0;
17 }

```

4.2.2 Reading Machine State

The following example demonstrates how to read the robot's joint states using the `neura::Robot::getLastMachineState()` function:

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4  #include <neura_cpp_api/robot.hpp>
5  #include <neura_cpp_api/utils.hpp>
6  #include <stdexcept>
7  #include <vector>
8
9  using Clock = std::chrono::high_resolution_clock;
10 using namespace std::chrono;
11
12 int main(int argc, char** argv)
13 {
14     size_t dof = 6;
15     unsigned robot_serial_number = 0; //Numerical number on serial number tag
16     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
17     if (!p_robot)
18     {
19         throw std::runtime_error("Could not instantiate robot interface no. " +
20                                 std::to_string(robot_serial_number));
21     }
22     return -1;
23 }
24     std::cout << "Created robot client no. "
25               << std::to_string(robot_serial_number) << std::endl;
26
27     for (int i = 0; i < 1000; i++)
28     {
29         uint64_t machine_state = 0;
30         if (p_robot->getLastMachineState(machine_state))
31         {
32             std::cout << "machine state/mode: " << std::to_string(machine_state)
33                       << std::endl;
34         }
35
36         std::this_thread::sleep_for(std::chrono::milliseconds(1));
37     }
38     return 0;
39 }

```

4.2.3 Reading Joint States

This example demonstrates how to use the *Robot* object to read joint states. It utilizes the `neura::Robot::getLastJointStates()` function:

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4  #include <neura_cpp_api/robot.hpp>
5  #include <neura_cpp_api/utils.hpp>
6  #include <stdexcept>
7  #include <vector>
8
9  using Clock = std::chrono::high_resolution_clock;
10 using namespace std::chrono;
11
12 int main(int argc, char** argv)
13 {
14     size_t dof = 6;
15     unsigned robot_serial_number = 0; //Numerical number on serial number tag
16     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
17     if (!p_robot)
18     {
19         throw std::runtime_error("Could not instantiate robot interface no. " +
20                                 std::to_string(robot_serial_number));
21         return -1;
22     }
23     std::cout << "Created robot client no. "
24               << std::to_string(robot_serial_number) << std::endl;
25
26     std::vector<double> pos_state;
27     std::vector<double> vel_state;
28     std::vector<double> torque_state;
29     double time;
30     for (int i = 0; i < 1000; i++)
31     {
32         if (p_robot->getLastJointStates(pos_state, vel_state, torque_state, time))
33         {
34             neura::utils::printState(pos_state, vel_state, torque_state, time);
35         }
36
37         std::this_thread::sleep_for(std::chrono::milliseconds(1));
38     }
39     return 0;
40 }
```

4.2.4 Reading Cartesian States

This example demonstrates how to use the *Robot* object to read Cartesian states (TCP position) using the `neura::Robot::getLastCartesianState()` function:

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4  #include <neura_cpp_api/robot.hpp>
5  #include <neura_cpp_api/utils.hpp>
6  #include <stdexcept>
7  #include <vector>
8
9  using Clock = std::chrono::high_resolution_clock;
10 using namespace std::chrono;
11
12 int main(int argc, char** argv)
13 {
14     size_t dof = 6;
15     unsigned robot_serial_number = 0; //Numerical number on serial number tag
16     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
17     if (!p_robot)
18     {
19         throw std::runtime_error("Could not instantiate robot interface no. " +
20                                 std::to_string(robot_serial_number));
21         return -1;
22     }
23     std::cout << "Created robot client no. "
24               << std::to_string(robot_serial_number) << std::endl;
25
26     std::vector<double> position;
27     std::vector<double> quaternion;
28     std::vector<double> trans_vel;
29     std::vector<double> rot_vel;
30
31     double time_buf;
32     for (int i = 0; i < 1000; i++)
33     {
34         if(!p_robot->getLastCartesianState(position, quaternion, trans_vel, rot_vel,
35         ↪time_buf))
36         {
37             std::cout << "Cartesian state : no new value" << std::endl;
38         }
39         else
40         {
41             std::cout << " Cartesian state : " << std::endl;
42             neura::utils::printState(position, quaternion, trans_vel, time_buf);
43         }
44
45         std::this_thread::sleep_for(std::chrono::milliseconds(1));
46     }
47     return 0;
48 }

```

4.2.5 SpeedJ

The following example demonstrates how to use the *Robot* object to execute SpeedJ (velocity joint control) using the `neura::Robot::speedJ()` function. This function requires the target joint velocities and accelerations for each joint, with vector lengths matching the degree of freedom of the robot.

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4  #include <neura_cpp_api/robot.hpp>
5  #include <neura_cpp_api/utils.hpp>
6  #include <stdexcept>
7  #include <vector>
8  #include <csignal>
9
10 using Clock = std::chrono::high_resolution_clock;
11 using namespace std::chrono;
12
13 // Pointer to store the lambda function
14 std::function<void(int)> signalHandlerWrapper;
15
16 // Wrapper function to call the stored lambda
17 void signalHandlerBridge(int signum) {
18     if (signalHandlerWrapper) {
19         signalHandlerWrapper(signum);
20     }
21 }
22
23 int main(int argc, char** argv)
24 {
25     size_t dof = 6;
26     unsigned robot_serial_number = 0; //Numerical number on serial number tag
27     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
28     if (!p_robot)
29     {
30         throw std::runtime_error("Could not instantiate robot interface no. " +
31                                 std::to_string(robot_serial_number));
32     }
33     return -1;
34 }
35
36 std::cout << "Created robot client no. "
37           << std::to_string(robot_serial_number) << std::endl;
38
39 signalHandlerWrapper = [&](int signum) {
40     std::cout << "Interrupt signal (" << signum << ") received.\n";
41     std::this_thread::sleep_for(std::chrono::milliseconds(10));
42     if (p_robot)
43     {
44         std::cout << "Stopping robot " << std::endl;
45         p_robot.reset();
46     }
47     std::this_thread::sleep_for(std::chrono::milliseconds(100));
48     exit(signum);
49 };

```

(continues on next page)

(continued from previous page)

```

48     std::signal(SIGINT, signalHandlerBridge);
49
50     p_robot->powerON(); //It is necessary to power on the robot through this command
51     ↪ before sending servo commands. (The robot will power off automatically if this
52     ↪ function is not called)
53     std::this_thread::sleep_for(std::chrono::seconds(1));
54
55     std::vector<double> vel_cmd(dof, 0.0);
56     std::vector<double> accel_cmd(dof, 0.0);
57     std::vector<double> pos_buf;
58     std::vector<double> vel_buf;
59     std::vector<double> accel_buf;
60     double time_buf;
61     double vel_add = 0.002;
62     for (int i = 0; i < 1000; i++)
63     {
64         if (p_robot->getLastJointStates(pos_buf, vel_buf, accel_buf, time_buf))
65         {
66             for (int j = 0; j < dof; j++)
67             {
68                 if (i < 500)
69                 {
70                     vel_cmd[j] = vel_buf[j] + vel_add; /// Take the last state and add 0.
71                     ↪ 002 rad/sec to the joint
72                 }
73                 else
74                 {
75                     vel_cmd[j] = vel_buf[j] - vel_add; /// Take the last state and
76                     ↪ subtract 0.02 rad/sec to the joint
77                 }
78             }
79             neura::utils::printState(pos_buf, vel_buf, accel_buf, time_buf);
80             int success = p_robot->speedJ(vel_cmd, accel_cmd);
81             std::cout << "speedj feedback: " << success << std::endl;
82         }
83     }
84
85     std::this_thread::sleep_for(std::chrono::milliseconds(1));
86
87     std::cout << "SpeedJ is finished " << std::endl;
88     p_robot->powerOFF();
89     std::this_thread::sleep_for(std::chrono::seconds(1));
90     return 0;
91 }

```

4.2.6 ServoJ

This section demonstrates the use of the *Robot* object to perform joint position control with the ServoJ command. This function uses the `neura::Robot::servoJ()` method. As an example, the interpolation library *Ruckig* is applied to achieve smooth trajectory generation. More information about *Ruckig* can be found on its official website: <https://ruckig.com/>. Alternative interpolation methods may also be integrated with the API.

The primary arguments required for *servoJ* include:

- **Target Position:** Desired joint positions.
- **Velocity:** Desired joint velocities.
- **Acceleration:** Desired joint accelerations.

All input vectors must match the degree of freedom of the robot.

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4  #include <neura_cpp_api/robot.hpp>
5  #include <neura_cpp_api/utils.hpp>
6  #include <stdexcept>
7  #include <vector>
8  #include <csignal>
9  #include <ruckig/ruckig.hpp>
10
11 using Clock = std::chrono::high_resolution_clock;
12 using namespace std::chrono;
13 using namespace ruckig;
14
15 // Helper function to convert std::vector to std::array
16 template <size_t N>
17 std::array<double, N> to_array(const std::vector<double>& vec)
18 {
19     std::array<double, N> arr = {};
20     std::copy_n(vec.begin(), N, arr.begin());
21     return arr;
22 }
23
24 // Helper function to convert std::array to std::vector
25 template <size_t N>
26 std::vector<double> to_vector(const std::array<double, N>& arr)
27 {
28     return std::vector<double>(arr.begin(), arr.end());
29 }
30
31 // Pointer to store the lambda function
32 std::function<void(int)> signalHandlerWrapper;
33
34 // Wrapper function to call the stored lambda
35 void signalHandlerBridge(int signum) {
36     if (signalHandlerWrapper) {
37         signalHandlerWrapper(signum);
38     }
39 }
```

(continues on next page)

(continued from previous page)

```

40
41 int main(int argc, char** argv)
42 {
43     const int dof = 6;
44     unsigned robot_serial_number = 0; //Numerical number on serial number tag
45     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
46     if (!p_robot)
47     {
48         throw std::runtime_error("Could not instantiate robot interface no. " +
49                                 std::to_string(robot_serial_number));
50         return -1;
51     }
52     std::cout << "Created robot client no. "
53               << std::to_string(robot_serial_number) << std::endl;
54
55     signalHandlerWrapper = [&](int signum) {
56         std::cout << "Interrupt signal (" << signum << ") received.\n";
57         std::this_thread::sleep_for(std::chrono::milliseconds(10));
58         if (p_robot)
59         {
60             std::cout << "Stopping robot " << std::endl;
61             p_robot.reset();
62         }
63         std::this_thread::sleep_for(std::chrono::milliseconds(100));
64         exit(signum);
65     };
66
67     std::signal(SIGINT, signalHandlerBridge);
68
69     Ruckig<dof> otg(0.001); // control cycle
70     InputParameter<dof> input;
71     OutputParameter<dof> output;
72
73     std::vector<double> pos_buf;
74     std::vector<double> vel_buf;
75     std::vector<double> accel_buf;
76     double time_buf;
77
78     for (int i = 0; i < 10; i++) /// Get the current joint state of the robot
79     {
80
81         if (p_robot->getLastJointStates(pos_buf, vel_buf, accel_buf, time_buf))
82         {
83             neura::utils::printState(pos_buf, vel_buf, accel_buf, time_buf);
84         }
85
86         std::this_thread::sleep_for(std::chrono::milliseconds(1));
87     }
88
89     p_robot->powerON(); //It is necessary to power on the robot through this command
↳ before sending servo commands. (The robot will power off automatically if this
↳ function is not called)

```

(continues on next page)

(continued from previous page)

```

90     std::this_thread::sleep_for(std::chrono::seconds(1));
91
92
93     std::array<double, dof> pos_cmd;
94     std::array<double, dof> zeros;
95
96     for(uint i = 0; i < dof; i++)
97     {
98         pos_cmd.at(i) = pos_buf.at(i) + 0.2; ///Add 0.2 radian to each joint angle
99         zeros.at(i) = 0;
100     }
101
102     input.current_position = to_array<dof>(pos_buf); /// Update the ruckig parameters_
↪ with current state
103     input.current_velocity = zeros; /// Assuming starting velocities to be 0
104     input.current_acceleration = zeros; /// Assuming starting velocities to be 0
105
106     input.target_position = pos_cmd;
107     input.target_velocity = zeros; /// Assuming end velocities to be 0
108     input.target_acceleration = zeros; /// Assuming end accelerations to be 0
109
110     std::array<double, dof> max_velocity;
111     max_velocity.fill(0.15);
112
113     std::array<double, dof> max_acceleration;
114     max_acceleration.fill(20.);
115
116     std::array<double, dof> max_jerk;
117     max_jerk.fill(2.);
118
119     input.max_velocity = max_velocity; ///Give the max velocities for interpolation
120     input.max_acceleration = max_acceleration; ///Give the max acceleration for_
↪ interpolation
121     input.max_jerk = max_jerk; ///Give the max jerk for interpolation
122
123     std::vector<double> target_position;
124     std::vector<double> target_velocity;
125     std::vector<double> target_acceleration;
126
127     while((otg.update(input, output) != Result::Finished))
128     {
129         input.current_position = output.new_position;
130         input.current_velocity = output.new_velocity;
131         input.current_acceleration = output.new_acceleration;
132
133         target_position = to_vector<dof>(input.current_position);
134         target_velocity = to_vector<dof>(input.current_velocity);
135         target_acceleration = to_vector<dof>(input.current_acceleration);
136
137         auto success = p_robot->servoJ(target_position, target_velocity, target_
↪ acceleration);
138

```

(continues on next page)

(continued from previous page)

```

139     std::this_thread::sleep_for(std::chrono::milliseconds(1));
140 }
141
142 p_robot->powerOFF();
143 std::this_thread::sleep_for(std::chrono::seconds(1));
144 return 0;
145 }

```

4.2.7 ServoX

This example demonstrates the use of the *Robot* object to perform Cartesian position control using the ServoX command. It uses the `neura::Robot::servoX()` method, with *Ruckig* as the interpolation library to enable smooth transitions between commands. For more information on *Ruckig*, refer to <https://ruckig.com/>. Users may also choose other interpolation libraries compatible with the API.

The arguments required for *servoX* are as follows:

- **Target Position:** A 7-element vector representing Cartesian translation and orientation in quaternion form (X, Y, Z, qw, qx, qy, qz).
- **Velocity:** A 6-element vector for Cartesian linear and angular velocities (x_dot, y_dot, z_dot, omega_x, omega_y, omega_z).
- **Acceleration:** A 6-element vector for Cartesian linear and angular accelerations (x_ddot, y_ddot, z_ddot, alpha_x, alpha_y, alpha_z).

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4  #include <neura_cpp_api/robot.hpp>
5  #include <neura_cpp_api/utils.hpp>
6  #include <stdexcept>
7  #include <vector>
8  #include <csignal>
9  #include <ruckig/ruckig.hpp>
10
11 using Clock = std::chrono::high_resolution_clock;
12 using namespace std::chrono;
13 using namespace ruckig;
14
15 // Helper function to convert std::vector to std::array
16 template <size_t N>
17 std::array<double, N> to_array(const std::vector<double>& vec)
18 {
19     std::array<double, N> arr = {};
20     std::copy_n(vec.begin(), N, arr.begin());
21     return arr;
22 }
23
24 // Helper function to convert std::array to std::vector
25 template <size_t N>
26 std::vector<double> to_vector(const std::array<double, N>& arr)
27 {

```

(continues on next page)

(continued from previous page)

```

28     return std::vector<double>(arr.begin(), arr.end());
29 }
30
31 // Pointer to store the lambda function
32 std::function<void(int)> signalHandlerWrapper;
33
34 // Wrapper function to call the stored lambda
35 void signalHandlerBridge(int signum) {
36     if (signalHandlerWrapper) {
37         signalHandlerWrapper(signum);
38     }
39 }
40
41 int main(int argc, char** argv)
42 {
43     const int cartesian_vector_length = 7; //X, Y, Z, qw, qx, qy, qt
44     unsigned robot_serial_number = 0; //Numerical number on serial number tag
45     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
46     if (!p_robot)
47     {
48         throw std::runtime_error("Could not instantiate robot interface no. " +
49                                 std::to_string(robot_serial_number));
50     }
51     return -1;
52     std::cout << "Created robot client no. "
53               << std::to_string(robot_serial_number) << std::endl;
54
55     signalHandlerWrapper = [&](int signum) {
56         std::cout << "Interrupt signal (" << signum << ") received.\n";
57         std::this_thread::sleep_for(std::chrono::milliseconds(10));
58         if (p_robot)
59         {
60             std::cout << "Stopping robot " << std::endl;
61             p_robot.reset();
62         }
63         std::this_thread::sleep_for(std::chrono::milliseconds(100));
64         exit(signum);
65     };
66
67     std::signal(SIGINT, signalHandlerBridge);
68
69     Ruckig<cartesian_vector_length> otg(0.001); // control cycle
70     InputParameter<cartesian_vector_length> input;
71     OutputParameter<cartesian_vector_length> output;
72
73     std::vector<double> position;
74     std::vector<double> quaternion;
75     std::vector<double> trans_vel;
76     std::vector<double> rot_vel;
77
78     double time_buf;
79     for (int i = 0; i < 10; i++)

```

(continues on next page)

(continued from previous page)

```

80     {
81         if(!p_robot->getLastCartesianState(position, quaternion, trans_vel, rot_vel,
↪time_buf))
82         {
83             std::cout << "Cartesian state : no new value" << std::endl;
84         }
85         else
86         {
87             neura::utils::printStats(position, quaternion, trans_vel, time_buf);
88         }
89         std::this_thread::sleep_for(std::chrono::milliseconds(1));
90     }
91
92     std::array<double, cartesian_vector_length> pos_cmd;
93     std::array<double, cartesian_vector_length> zeros = {0};
94
95     std::array<double, cartesian_vector_length> current_position;
96     for(uint i = 0; i < 3; i++)
97     {
98         pos_cmd.at(i) = position.at(i) + 0.1; ///Add 0.1 m in X, Y and Y direction
99         current_position.at(i) = position.at(i);
100     }
101
102     for(uint i = 0; i < 4; i++)
103     {
104         pos_cmd.at(i + 3) = quaternion.at(i); /// No change in orientation
105         current_position.at(i + 3) = quaternion.at(i);
106     }
107
108     p_robot->powerON(); ///It is necessray to power on the robot through this command
↪before sending servo commands. (The robot will power off automatically if this
↪function is not called)
109     std::this_thread::sleep_for(std::chrono::seconds(1));
110
111     input.current_position = current_position; /// Update the ruckig parameters with
↪current state
112     input.current_velocity = zeros; /// Assuming starting velocities to be 0
113     input.current_acceleration = zeros; /// Assuming starting velocities to be 0
114
115     input.target_position = pos_cmd;
116     input.target_velocity = zeros; /// Assuming end velocities to be 0
117     input.target_acceleration = zeros; /// Assuming end accelerations to be 0
118
119     input.max_velocity = {0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5}; ///Give the max velocities
↪for interpolation
120     input.max_acceleration = {2., 2., 2., 2., 2., 2., 2.}; ///Give the max acceleration
↪for interpolation
121     input.max_jerk = {20., 20., 20., 20., 20., 20., 20.}; ///Give the max jerk for
↪interpolation
122
123     std::vector<double> target_position;
124     std::vector<double> target_velocity;

```

(continues on next page)

(continued from previous page)

```

125     std::vector<double> target_acceleration;
126
127     target_velocity.resize(6, 0.); ///Since ruckig does not provide rotational velocity,
    ↪from quaternion, we can input angular velocity feedforward as 0
128     target_acceleration.resize(6, 0.); ///Since ruckig does not provide rotational
    ↪acceleration from quaternion, we can input angular acceleration feedforward as 0
129
130     double gain_parameter = 2.;
131
132     while((otg.update(input, output) != Result::Finished))
133     {
134         input.current_position = output.new_position;
135         input.current_velocity = output.new_velocity;
136         input.current_acceleration = output.new_acceleration;
137
138         target_position = to_vector<cartesian_vector_length>(input.current_position);
139
140         for(uint i = 0; i < 3; i++) /// Updating target translation velocity and
    ↪accelerations
141         {
142             target_velocity.at(i) = output.new_velocity.at(i);
143             target_acceleration.at(i) = output.new_acceleration.at(i);
144         }
145
146         auto success = p_robot->servoX(target_position, target_velocity, target_
    ↪acceleration, gain_parameter);
147
148         std::this_thread::sleep_for(std::chrono::milliseconds(1));
149     }
150
151     p_robot->powerOFF();
152     std::this_thread::sleep_for(std::chrono::seconds(1));
153
154     return 0;
155 }

```

4.2.8 Move Linear Online

This section describes how to use the *Robot* object to control Cartesian positioning through linear movements with the `MoveLinearOnline` command. This function utilizes both the `neura::Robot::moveLinearOnline()` and `neura::Robot::stopMoveLinearOnline()` methods. Internally, an interpolation process is executed to generate smooth motion commands between specified targets.

The following parameters are required for *moveLinearOnline*:

- **Target Position:** A 7-element vector containing translation and orientation (X, Y, Z, qw, qx, qy, qz).
- **Maximum Velocities:** A 6-element vector indicating maximum translational and rotational velocities.
- **Maximum Acceleration:** A 6-element vector indicating maximum translational and rotational accelerations.

Each of these parameters supports smooth, continuous control of the robot along a specified linear path.

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4  #include <neura_cpp_api/robot.hpp>
5  #include <neura_cpp_api/utils.hpp>
6  #include <stdexcept>
7  #include <vector>
8  #include <csignal>
9
10 using Clock = std::chrono::high_resolution_clock;
11 using namespace std::chrono;
12
13 // Pointer to store the lambda function
14 std::function<void(int)> signalHandlerWrapper;
15
16 // Wrapper function to call the stored lambda
17 void signalHandlerBridge(int signum) {
18     if (signalHandlerWrapper) {
19         signalHandlerWrapper(signum);
20     }
21 }
22
23 int main(int argc, char** argv)
24 {
25     const int cartesian_vector_length = 7; //X, Y, Z, qw, qx, qy, qt
26     unsigned robot_serial_number = 0; //Numerical number on serial number tag
27     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
28     if (!p_robot)
29     {
30         throw std::runtime_error("Could not instantiate robot interface no. " +
31                                 std::to_string(robot_serial_number));
32         return -1;
33     }
34     std::cout << "Created robot client no. "
35               << std::to_string(robot_serial_number) << std::endl;
36
37     // Lambda function to handle the signal
38     signalHandlerWrapper = [&](int signum) {
39         std::cout << "Interrupt signal (" << signum << ") received.\n";
40         std::this_thread::sleep_for(std::chrono::milliseconds(10));
41         if (p_robot)
42         {
43             std::cout << "Stopping robot " << std::endl;
44             p_robot->stopMoveLinearOnline();
45             p_robot.reset();
46         }
47         std::this_thread::sleep_for(std::chrono::milliseconds(100));
48         exit(signum);
49     };
50
51     std::signal(SIGINT, signalHandlerBridge);
52
53     std::vector<double> position;

```

(continues on next page)

(continued from previous page)

```

54     std::vector<double> quaternion;
55     std::vector<double> trans_vel;
56     std::vector<double> rot_vel;
57
58     double time_buf;
59     for (int i = 0; i < 10; i++)
60     {
61         if(!p_robot->getLastCartesianState(position, quaternion, trans_vel, rot_vel,
↪time_buf))
62         {
63             std::cout << "Cartesian state : no new value" << std::endl;
64         }
65         else
66         {
67             neura::utils::printState(position, quaternion, trans_vel, time_buf);
68         }
69         std::this_thread::sleep_for(std::chrono::milliseconds(1));
70     }
71
72     std::vector<double> pos_cmd;
73     pos_cmd.resize(cartesian_vector_length);
74
75     for(uint i = 0; i < 3; i++)
76     {
77         pos_cmd.at(i) = position.at(i) + 0.1; ///Add 0.1 m in X, Y and Y direction
78     }
79
80     for(uint i = 0; i < 4; i++)
81     {
82         pos_cmd.at(i + 3) = quaternion.at(i); /// No change in orientation
83     }
84
85     p_robot->powerON();
86     std::this_thread::sleep_for(std::chrono::seconds(1));
87
88     std::vector<double> max_velocity = {0.05, 0.05, 0.05, 0.05, 0.05, 0.05};
89     std::vector<double> max_acceleration = {2., 2., 2., 2., 2., 2.};
90
91     auto success = p_robot->moveLinearOnline(pos_cmd, max_velocity, max_acceleration);
92
93     std::this_thread::sleep_for(std::chrono::seconds(2)); // Run motion for 2 seconds
94
95     for(uint i = 0; i < 3; i++)
96     {
97         pos_cmd.at(i) = position.at(i); ///Set the starting position as target to change
↪targets online
98     }
99     for(uint i = 0; i < 4; i++)
100     {
101         pos_cmd.at(i + 3) = quaternion.at(i);
102     }
103     success = p_robot->moveLinearOnline(pos_cmd, max_velocity, max_acceleration);

```

(continues on next page)

(continued from previous page)

```

104     std::this_thread::sleep_for(std::chrono::seconds(2)); // Run motion for 2 seconds
105
106
107     std::cout << "Stopping robot" << std::endl;
108     p_robot->stopMoveLinearOnline(); // Stop the motion in between if its not complete
109     std::this_thread::sleep_for(std::chrono::seconds(2));
110
111     p_robot->powerOFF();
112     std::this_thread::sleep_for(std::chrono::seconds(1));
113
114     return 0;
115 }

```

4.2.9 Joint Limits

Here's how to use *Robot* object to get joint limits of the robot set through GUI. It uses the `neura::Robot::getCurrentJointLimits()` to get the joint position and velocity limits. The first elements are positive position limits for each joint (in radians), next elements are negative position limits for each joint (in radians), next elements are positive velocity limits for each joint (in radians/second) and last set of elements are negative velocity limits for each joint (in radians/second).

```

1  #include <atomic>
2  #include <chrono>
3  #include <iostream>
4  #include <stdexcept>
5  #include <thread>
6  #include <vector>
7
8  #include <neura_cpp_api/robot.hpp>
9
10 void printVector(const std::vector<double>& vec)
11 {
12     std::cout << "Printing Vector: " << std::endl;
13     for(const auto &x: vec)
14     {
15         std::cout << x << " ";
16     }
17     std::cout << std::endl;
18 }
19
20 int main(int argc, char** argv)
21 {
22     neura::Robot robot(0); //Numerical number on serial number tag
23
24     std::vector<double> joint_limits;
25
26     auto success = robot.getCurrentJointLimits(joint_limits);
27     std::cout << "Joint Limits" << std::endl;
28     printVector(joint_limits);
29
30     return 0;

```

(continues on next page)

(continued from previous page)

31 }

4.2.10 Cartesian Limits

Here's how to use *Robot* object to get cartesian limits of the end effector set through GUI. It uses the `neura::Robot::getCurrentCartesianLimits()` to get the cartesian position and velocity limits. The first three elements are positive X,Y,Z position limits (in meters), next 3 elements are negative X,Y,Z position limits (in meters), next element is translation velocity (in meters/second), next element is rotational velocity (in radians/second).

```

1  #include <atomic>
2  #include <chrono>
3  #include <iostream>
4  #include <stdexcept>
5  #include <thread>
6  #include <vector>
7
8  #include <neura_cpp_api/robot.hpp>
9
10 void printVector(const std::vector<double>& vec)
11 {
12     std::cout << "Printing Vector: " << std::endl;
13     for(const auto &x: vec)
14     {
15         std::cout << x << " ";
16     }
17     std::cout << std::endl;
18 }
19
20 int main(int argc, char** argv)
21 {
22     neura::Robot robot(0); //Numerical number on serial number tag
23
24     std::vector<double> cartesian_limits;
25
26     auto success = robot.getCurrentCartesianLimits(cartesian_limits);
27     std::cout << "Cartesian Limits" << std::endl;
28     printVector(cartesian_limits);
29
30     return 0;
31 }
```

4.2.11 Forward Kinematics

Here's how to use *Robot* object to get forward kinematics pose of a set of joint angles. It uses the `neura::Robot::getForwardKinematics()` to get the forward kinematics pose. The first argument is set of joint angles(in radians) with length equal to degree of freedom, second argument will be the result sent through reference. If tool is added using GUI, forward kinematics will be automatically updated with latest tool parameters.

```

1  #include <atomic>
2  #include <chrono>
3  #include <iostream>
4  #include <stdexcept>
5  #include <thread>
6  #include <vector>
7
8  #include <neura_cpp_api/robot.hpp>
9
10 void printVector(const std::vector<double>& vec)
11 {
12     std::cout << "Printing Vector: " << std::endl;
13     for(const auto &x: vec)
14     {
15         std::cout << x << " ";
16     }
17     std::cout << std::endl;
18 }
19
20 int main(int argc, char** argv)
21 {
22     neura::Robot robot(0); //Numerical number on serial number tag
23
24     std::vector<double>fk_request = {0., 0., 0., 1.57, 0.,1.57, 0.}; //Done for 7 DOF
    ↪MAIRA7M
25     std::vector<double>fk_feedback;
26     fk_feedback.resize(7);
27
28     auto success = robot.getForwardKinematics(fk_request, fk_feedback);
29
30     std::cout << "Success " << success << std::endl;
31     printVector(fk_feedback);
32
33     return 0;
34 }

```

4.2.12 Inverse Kinematics

Here's how to use *Robot* object to get inverse kinematics (set of joint angles) for a specific pose. It uses the `neura::Robot::getInverseKinematics()` to get the inverse kinematics joint angles. The first argument is pose (X,Y,Z,qw,qx,qy,qz) (translation in meters), second argument will be the result sent through reference. If tool is added using GUI, forward kinematics will be automatically updated with latest tool parameters.

```

1  #include <atomic>
2  #include <chrono>

```

(continues on next page)

(continued from previous page)

```

3  #include <iostream>
4  #include <stdexcept>
5  #include <thread>
6  #include <vector>
7
8  #include <neura_cpp_api/robot.hpp>
9
10 void printVector(const std::vector<double>& vec)
11 {
12     std::cout << "Printing Vector: " << std::endl;
13     for(const auto &x: vec)
14     {
15         std::cout << x << " ";
16     }
17     std::cout << std::endl;
18 }
19
20 int main(int argc, char** argv)
21 {
22     neura::Robot robot(0); //Numerical number on serial number tag
23
24     std::vector<double>fk_request = {0., 0., 0., 1.57, 0.,1.57, 0.}; //Done for 7 DOF
    ↪MAIRA7M
25     std::vector<double>fk_feedback;
26     fk_feedback.resize(7);
27
28     auto success = robot.getForwardKinematics(fk_request, fk_feedback);
29
30     std::cout << "Success " << success << std::endl;
31     printVector(fk_feedback);
32
33     //Verify the FK solution through IK
34     success = robot.getInverseKinematics(fk_feedback, ik_feedback);
35
36     printVector(ik_feedback);
37     std::cout << "Success " << success << std::endl;
38
39     return 0;
40 }

```

4.2.13 SpeedX

Here's how to use *Robot* object to run SpeedX (cartesian velocity control). It uses the `neura::Robot::speedX()`. The arguments are cartesian target velocity and acceleration at each timestep with length of argument as six (three translation velocity/acceleration, three rotational velocity/acceleration).

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4  #include <stdexcept>
5  #include <vector>

```

(continues on next page)

(continued from previous page)

```

6  #include <csignal>
7
8  #include <neura_cpp_api/robot.hpp>
9  #include <neura_cpp_api/utils.hpp>
10
11 using Clock = std::chrono::high_resolution_clock;
12 using namespace std::chrono;
13
14
15 // Pointer to store the lambda function
16 std::function<void(int)> signalHandlerWrapper;
17
18 // Wrapper function to call the stored lambda
19 void signalHandlerBridge(int signum) {
20     if (signalHandlerWrapper) {
21         signalHandlerWrapper(signum);
22     }
23 }
24
25 int main(int argc, char** argv)
26 {
27     int cart_velocity_vector_length = 6;
28     unsigned robot_serial_number = 0; //Numerical number on serial number tag
29     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
30     if (!p_robot)
31     {
32         throw std::runtime_error("Could not instantiate robot interface no. " +
33                                 std::to_string(robot_serial_number));
34         return -1;
35     }
36     std::cout << "Created robot client no. "
37               << std::to_string(robot_serial_number) << std::endl;
38
39     signalHandlerWrapper = [&](int signum) {
40         std::cout << "Interrupt signal (" << signum << ") received.\n";
41         std::this_thread::sleep_for(std::chrono::milliseconds(10));
42         if (p_robot)
43         {
44             std::cout << "Stopping robot " << std::endl;
45             p_robot.reset();
46         }
47         std::this_thread::sleep_for(std::chrono::milliseconds(100));
48         exit(signum);
49     };
50
51     std::signal(SIGINT, signalHandlerBridge);
52
53     std::vector<double> vel_cmd(cart_velocity_vector_length, 0.0);
54     std::vector<double> accel_cmd(cart_velocity_vector_length, 0.0);
55
56     std::vector<double> position;
57     std::vector<double> quaternion;

```

(continues on next page)

(continued from previous page)

```

58     std::vector<double> trans_vel;
59     std::vector<double> rot_vel;
60     double time_buf;
61
62     p_robot->powerON();
63
64     std::this_thread::sleep_for(std::chrono::seconds(1));
65     double vel = 0.1;
66     for (int i = 0; i < 3000; i++)
67     {
68         if (p_robot->getLastCartesianState(position, quaternion, trans_vel, rot_vel,
69 ↪time_buf))
70         {
71             for (int j = 0; j < 2; j++) //giving velocity to X and Y direction
72             {
73                 if (i < 1500)
74                 {
75                     vel_cmd[j] = vel; /// Give velocity of 0.1 m/s
76                 }
77                 else
78                 {
79                     vel_cmd[j] = -vel; /// Give velocity of -0.1 m/s
80                 }
81             }
82             neura::utils::printState(position, quaternion, trans_vel, time_buf);
83             int success = p_robot->speedX(vel_cmd, accel_cmd);
84             std::cout << "speedX feedback: " << success << std::endl;
85         }
86
87         std::this_thread::sleep_for(std::chrono::milliseconds(1));
88     }
89
90     p_robot->powerOFF();
91     std::this_thread::sleep_for(std::chrono::seconds(1));
92
93     return 0;
94 }

```

4.2.14 Reading Force Torque Sensor States

This example demonstrates how to use the *Robot* object to read the external wrench (forces and moments) coming from the force torque sensor connected to flange of the robot. It utilizes the `neura::Robot::getExternalWrench()` function:

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4  #include <neura_cpp_api/robot.hpp>
5  #include <neura_cpp_api/utils.hpp>
6  #include <stdexcept>

```

(continues on next page)

(continued from previous page)

```

7  #include <vector>
8  using Clock = std::chrono::high_resolution_clock;
9  using namespace std::chrono;
10
11 void printVector(const std::vector<double>& vector, std::string name)
12 {
13     std::cout << "Printing " << name << std::endl;
14
15     for(uint i = 0; i < vector.size(); i++)
16     {
17         std::cout << vector.at(i) << " ";
18     }
19     std::cout << std::endl;
20 }
21
22 int main(int argc, char** argv)
23 {
24     size_t dof = 7;
25     unsigned robot_serial_number = 0; //Numerical number on serial number tag
26     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
27     if (!p_robot)
28     {
29         throw std::runtime_error("Could not instantiate robot interface no. " +
30                                 std::to_string(robot_serial_number));
31         return -1;
32     }
33     std::cout << "Created robot client no. "
34               << std::to_string(robot_serial_number) << std::endl;
35     std::vector<double> joint_angles;
36     std::vector<double> joint_velocities;
37     std::vector<double> joint_torques;
38     double timestamp;
39     std::vector<double> external_wrench;
40
41     double time;
42     while(true)
43     {
44         if(p_robot->getLastJointStates(joint_angles, joint_velocities, joint_torques,
45 ↪ timestamp))
46         {
47             printVector(joint_angles, "joint_angles");
48             printVector(joint_torques, "joint_torques");
49         }
50
51         if(p_robot->getExternalWrench(external_wrench))
52         {
53             printVector(external_wrench, "external_wrench");
54         }
55
56         std::this_thread::sleep_for(std::chrono::milliseconds(1));
57     }
58     return 0;

```

(continues on next page)

(continued from previous page)

```

58 }

```

4.2.15 Trajectory Scaling Factor

Here's how to use *Robot* object to get the trajectory scaling factor. It uses the `neura::Robot::getTrajectoryScalingFactor()`. The return value would be between 0 & 1.

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4
5  #include <neura_cpp_api/robot.hpp>
6  #include <neura_cpp_api/utils.hpp>
7  #include <stdexcept>
8  #include <vector>
9  #include <csignal>
10
11 using Clock = std::chrono::high_resolution_clock;
12 using namespace std::chrono;
13
14 // Pointer to store the lambda function
15 std::function<void(int)> signalHandlerWrapper;
16
17 // Wrapper function to call the stored lambda
18 void signalHandlerBridge(int signum) {
19     if (signalHandlerWrapper) {
20         signalHandlerWrapper(signum);
21     }
22 }
23
24 int main(int argc, char** argv)
25 {
26     size_t dof = 6;
27     unsigned robot_serial_number = 0; //Numerical number on serial number tag
28     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
29     if (!p_robot)
30     {
31         throw std::runtime_error("Could not instantiate robot interface no. " +
32                                 std::to_string(robot_serial_number));
33         return -1;
34     }
35     std::cout << "Created robot client no. "
36               << std::to_string(robot_serial_number) << std::endl;
37
38     signalHandlerWrapper = [&](int signum) {
39         std::cout << "Interrupt signal (" << signum << ") received.\n";
40         std::this_thread::sleep_for(std::chrono::milliseconds(10));
41         if (p_robot)
42         {
43             std::cout << "Stopping robot " << std::endl;
44             p_robot.reset();

```

(continues on next page)

(continued from previous page)

```

45     }
46     std::this_thread::sleep_for(std::chrono::milliseconds(100));
47     exit(signum);
48     };
49
50     std::signal(SIGINT, signalHandlerBridge);
51
52     p_robot->powerON(); //It is necessary to power on the robot through this command
↳ before sending servo commands. (The robot will power off automatically if this
↳ function is not called)
53     std::this_thread::sleep_for(std::chrono::seconds(1));
54
55     std::vector<double> vel_cmd(dof, 0.0);
56     std::vector<double> accel_cmd(dof, 0.0);
57     std::vector<double> pos_buf;
58     std::vector<double> vel_buf;
59     std::vector<double> accel_buf;
60     double time_buf;
61     double vel_add = 0.002;
62
63     double trajectory_scaling_factor = 0.;
64     for (int i = 0; i < 1000; i++)
65     {
66         if (p_robot->getLastJointStates(pos_buf, vel_buf, accel_buf, time_buf))
67         {
68             for (int j = 0; j < dof; j++)
69             {
70                 if (i < 500)
71                 {
72                     vel_cmd[j] = vel_buf[j] + vel_add; /// Take the last state and add 0.
↳ 002 rad/sec to the joint
73                 }
74                 else
75                 {
76                     vel_cmd[j] = vel_buf[j] - vel_add; /// Take the last state and
↳ subtract 0.02 rad/sec to the joint
77                 }
78             }
79         }
80         neura::utils::printState(pos_buf, vel_buf, accel_buf, time_buf);
81         int success = p_robot->speedJ(vel_cmd, accel_cmd);
82         std::cout << "speedj feedback: " << success << std::endl;
83
84         trajectory_scaling_factor = p_robot->getTrajectoryScalingFactor();
85         std::cout << "trajectory_scaling_factor: " << trajectory_scaling_factor <<
↳ std::endl;
86
87     }
88
89     std::this_thread::sleep_for(std::chrono::milliseconds(1));
90 }
91

```

(continues on next page)

(continued from previous page)

```

92     std::cout << "SpeedJ is finsihed " << std::endl;
93     p_robot->powerOFF();
94     std::this_thread::sleep_for(std::chrono::seconds(1));
95     return 0;
96 }

```

4.2.16 Error Codes

Here's how to use *Robot* object to get the error code. It uses the `neura::Robot::getErrorCode()`. The return value can be used in the program to pause trajectory calculations.

```

1  #include <chrono>
2  #include <iostream>
3  #include <memory>
4  #include <neura_cpp_api/robot.hpp>
5  #include <neura_cpp_api/utils.hpp>
6  #include <stdexcept>
7  #include <vector>
8  #include <csignal>
9
10 using Clock = std::chrono::high_resolution_clock;
11 using namespace std::chrono;
12
13 // Pointer to store the lambda function
14 std::function<void(int)> signalHandlerWrapper;
15
16 // Wrapper function to call the stored lambda
17 void signalHandlerBridge(int signum) {
18     if (signalHandlerWrapper) {
19         signalHandlerWrapper(signum);
20     }
21 }
22
23 int main(int argc, char** argv)
24 {
25     const int cartesian_vector_length = 7; //X, Y, Z, qw, qx, qy, qt
26     unsigned robot_serial_number = 0; //Numerical number on serial number tag
27     auto p_robot = std::make_unique<neura::Robot>(robot_serial_number);
28     if (!p_robot)
29     {
30         throw std::runtime_error("Could not instantiate robot interface no. " +
31                                 std::to_string(robot_serial_number));
32         return -1;
33     }
34     std::cout << "Created robot client no. "
35               << std::to_string(robot_serial_number) << std::endl;
36
37     // Lambda function to handle the signal
38     signalHandlerWrapper = [&](int signum) {
39         std::cout << "Interrupt signal (" << signum << ") received.\n";
40         std::this_thread::sleep_for(std::chrono::milliseconds(10));

```

(continues on next page)

(continued from previous page)

```

41     if (p_robot)
42     {
43         std::cout << "Stopping robot " << std::endl;
44         p_robot->stopMoveLinearOnline();
45         p_robot.reset();
46     }
47     std::this_thread::sleep_for(std::chrono::milliseconds(100));
48     exit(signum);
49 };
50
51 std::signal(SIGINT, signalHandlerBridge);
52
53 std::vector<double> position;
54 std::vector<double> quaternion;
55 std::vector<double> trans_vel;
56 std::vector<double> rot_vel;
57
58 double time_buf;
59 for (int i = 0; i < 10; i++)
60 {
61     if(!p_robot->getLastCartesianState(position, quaternion, trans_vel, rot_vel,
62 ↪time_buf))
63     {
64         std::cout << "Cartesian state : no new value" << std::endl;
65     }
66     else
67     {
68         neura::utils::printState(position, quaternion, trans_vel, time_buf);
69     }
70     std::this_thread::sleep_for(std::chrono::milliseconds(1));
71 }
72
73 std::vector<double> pos_cmd;
74 pos_cmd.resize(cartesian_vector_length);
75
76 for(uint i = 0; i < 3; i++)
77 {
78     pos_cmd.at(i) = position.at(i) + 0.1; ///Add 0.1 m in X, Y and Y direction
79 }
80
81 for(uint i = 0; i < 4; i++)
82 {
83     pos_cmd.at(i + 3) = quaternion.at(i); /// No change in orientation
84 }
85
86 p_robot->powerON();
87 std::this_thread::sleep_for(std::chrono::seconds(1));
88
89 std::vector<double> max_velocity = {0.05, 0.05, 0.05, 0.05, 0.05, 0.05};
90 std::vector<double> max_acceleration = {2., 2., 2., 2., 2., 2.};
91
92 auto success = p_robot->moveLinearOnline(pos_cmd, max_velocity, max_acceleration);

```

(continues on next page)

(continued from previous page)

```

92
93     int error_code = 0;
94     int iteration = 2000;
95
96     for(int i = 0; i < iteration; i++) // Run motion for 2 seconds
97     {
98         error_code = p_robot->getErrorCode();
99         std::cout << "Error code: " << error_code << std::endl;
100        std::this_thread::sleep_for(std::chrono::milliseconds(1));
101    }
102
103    for(uint i = 0; i < 3; i++)
104    {
105        pos_cmd.at(i) = position.at(i); ///Set the starting position as target to change_
↪targets online
106    }
107    for(uint i = 0; i < 4; i++)
108    {
109        pos_cmd.at(i + 3) = quaternion.at(i);
110    }
111    success = p_robot->moveLinearOnline(pos_cmd, max_velocity, max_acceleration);
112
113    std::this_thread::sleep_for(std::chrono::seconds(2)); // Run motion for 2 seconds
114
115    std::cout << "Stopping robot" << std::endl;
116    p_robot->stopMoveLinearOnline(); // Stop the motion in between if its not complete
117    std::this_thread::sleep_for(std::chrono::seconds(2));
118
119    p_robot->powerOFF();
120    std::this_thread::sleep_for(std::chrono::seconds(1));
121
122    return 0;
123 }

```

4.3 Guide for integrating API with custom projects

To ensure compatibility with the Neura API, incorporate the following steps in your *CMakeLists.txt* file. You may adjust other parts of the configuration as needed, but **make sure** to include the specified components.

4.3.1 Required Configuration Steps

1. **Set the C++ Standard** The Neura API requires C++17. Ensure your *CMakeLists.txt* file includes the following lines:

```

1 # Set C++ standard to 17
2 set(CMAKE_CXX_STANDARD 17)
3 set(CMAKE_CXX_STANDARD_REQUIRED ON)

```

2. **Link Required Libraries** Link the Neura API and its dependencies using the following code. The example below loops through your project's examples and links the necessary libraries:

```

1  # Loop through the EXAMPLES list to create executables and link libraries
2  add_executable(${example} ${example}.cpp)
3  target_link_libraries(${example} neuro_cpp_api dds_wrapper nrti_msgs_fastdds_
    ↪fastrtps fastcdr ruckig::ruckig)
4  #ruckig may not be needed for every example but it is needed for all the examples_
    ↪provided by Neura.

```

4.3.2 Sample CMakeLists.txt

Here is an example of a *CMakeLists.txt* that integrates the Neura API into your project. This *CMakeLists.txt* file provides the necessary configuration to integrate the Neura C++ API into a project. It sets the required C++17 standard and links essential dependencies. The file then goes through a list of example files, building an executable for each example, and ensures proper installation of the compiled binaries to the standard binary directory.

```

1  cmake_minimum_required(VERSION 3.10)
2
3  # Define project name
4  project(neura_cpp_api_examples)
5
6  # Set C++ standard to 17
7  set(CMAKE_CXX_STANDARD 17)
8  set(CMAKE_CXX_STANDARD_REQUIRED ON)
9
10 # Include the standard installation directories
11 include(GNUInstallDirs)
12
13 # Find Ruckig library
14 find_package(ruckig REQUIRED)
15 IF (ruckig_FOUND)
16     message(STATUS "${BoldGreen}[OK] ${ColourReset}Ruckig found. Version: ${ruckig_
    ↪VERSION}\n")
17     include_directories(ruckig)
18 ELSE()
19     message(FATAL_ERROR "${BoldRed}[NOT OK] ${ColourReset}ruckig NOT found.\n")
20 ENDIF()
21
22 # Define examples and link libraries
23 set(EXAMPLES example1 example2) # Replace with your actual executable names
24 foreach(example ${EXAMPLES})
25     add_executable(${example} ${example}.cpp)
26     target_link_libraries(${example} neuro_cpp_api dds_wrapper nrti_msgs_fastdds_
    ↪fastrtps fastcdr)
27 endforeach()
28
29 # Install executables to the standard binary directory
30 install(TARGETS ${EXAMPLES} DESTINATION ${CMAKE_INSTALL_BINDIR})

```

ROBOT SAFETY CONFIGURATIONS

Neura Cobots can be operated in various safety configurations and speeds such as Industrial Mode (Full Speed Mode), Cobot Mode. This section describes how to activate each of these modes and how does it affect the behaviour of Neura C++ API.

5.1 Safety Configurations

1. FullSpeed Mode or Industrial Mode

In this mode, the robot can be run at the maximum limits set through GUI. Approaching the limits will slow down the robot and scaling factor would be updated to account for this slow down. This factor can be accessed through `neura::Robot::getTrajectoryScalingFactor()`.

2. Cobot Mode

In this mode, the robot's maximum linear speed is **250 mm/s**. Approaching this limits will slow down the robot and scaling factor would be updated to account for this slow down. This factor can be accessed through `neura::Robot::getTrajectoryScalingFactor()`.

3. Safeguard Mode/Stop

In this mode, the robot will be in a paused state. This can be triggered by any external sensor such as Safety Curtains. The scaling factor would be 0 in this case.

5.2 Configuring and Switching Between Safety Modes

5.2.1 Lara

1. Bridge J01

This bridge is used for safeguard stop. If this bridge is disconnected or not present, the robot will be in paused state.

Important: When this bridge is connected again, white button has to be pressed on the Control Box for the robot to resume motion.

2. Bridge J02

This bridge is used for Cobot Mode or reduced mode. If this bridge is not present or disconnected, the robot's maximum linear speed would be changed to **250 mm/s**.

3. Brige J03

This bridge is used for Industrial mode or Full speed mode. If this bridge is not present or disconnected, the robot's maximum linear speed would be changed to **250 mm/s**.

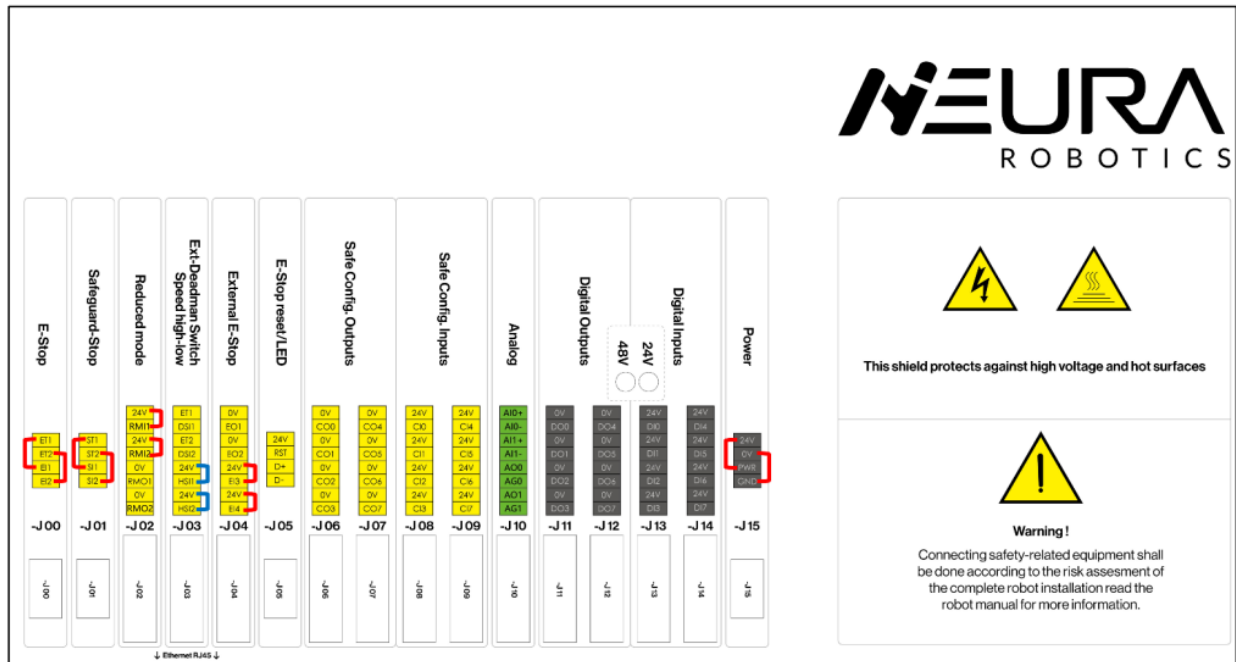


Fig. 1: Bridges present in LARA control box.

Important: All the three bridges should be connected for the robot to move in fullspeed within the limits set from GUI.

5.2.2 MAiRA

1. SDI10 and SDI11 (Reduced Mode/Cobot Mode)

This bridge is used for Cobot Mode or reduced mode. If this bridge is not present or disconnected, the robot's maximum linear speed would be changed to **250 mm/s**. There are two inputs for cobot mode, if any one of them is not present, cobot mode will be activated.

2. SDI3 and SDI8 (Protective Stop)

This bridge is used for safeguard stop. If this bridge is disconnected or not present, the robot will be in paused state. There are two inputs for protective stop, if any one of them is not present, protective stop will be activated.

3. Fullspeed Mode

When all the above mentioned bridges are connected, then the robot can move with fullspeed within the limits set in GUI.

Important: When this bridge is connected again, white button has to be pressed on the Control Box for the robot to resume motion.

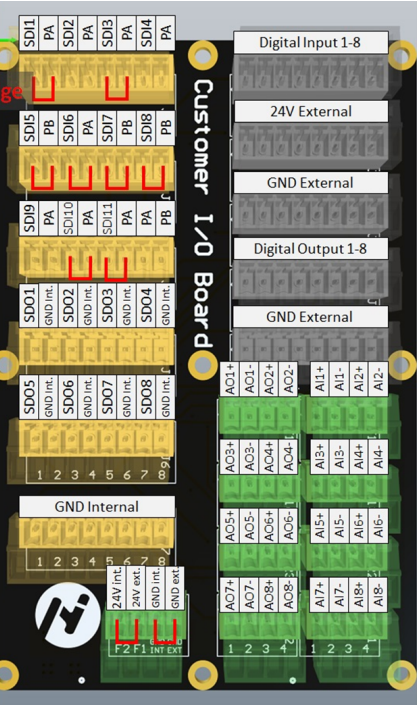
Safe Input	Pulse	PLC-S	Function					Safe Output	PLC-S	Function
SDI1	PA	IQI00	E-Stop SS1					SDO1	IQQ11	System Emergency Stop (SS1)
SDI2	PA	IQI02	48V Enable Button					SDO2	IQQ12	System Error
SDI3	24V/PA	IQI05	Safe Guard Stop 2					SDO3	IQQ13	Robot Moving
SDI4	PA	IQI07	Safe Guard Reset					SDO4	IQQ14	Safe Guard Stop active (SS2)
SDI5	PB	IQI01	E-Stop SS1					SDO5	IQQ15	Reduced Mode (SLS)
SDI6	24V/PA	IQI03	Safe Guard Stop 1					SDO6	IQQ16	Not Reduced Mode (not SLS)
SDI7	24V/PB	IQI04	Safe Guard Stop 1					SDO7	IQQ17	Safe Home (Home Pos.)
SDI8	24V/PB	IQI06	Safe Guard Stop 2					SDO8	IQQ18	Robot Not Stopping
SDI9	PA	IQI08	Zero G							
SDI10	PA	IQI09	Reduced Mode (SLS)							
SDI11	PA	IQI10	Reduced Mode (SLS)							

Fig. 2: Bridges present in MAiRA control box.

CHANGELOG AND VERSIONING INFORMATION

6.1 1.0 (13-11-2024)

Initial release of the Neura C++ API. This version provides a robust, extendable, and robot-agnostic interface for Neura's control software, specifically designed for application developers. Key features of this release include:

- Four real-time (1 kHz) control interfaces:
 - Joint position commands
 - Joint velocity commands
 - Cartesian pose commands
 - Cartesian velocity commands
- Access to 1 kHz measurement data:
 - Joint position, velocity and torque
 - Cartesian position and velocity
 - Operational limits for joint and cartesian values set on the GUI
 - External wrench (forces and moments) from force torque sensor
- Control over robot power state:
 - Power on or off the robot
- Integration with the Robot Model Library:
 - Forward kinematics specified sets of robot joint states
 - Inverse kinematics for defined cartesian poses

This release also includes comprehensive documentation and examples to facilitate setup, integration, and usage.

6.2 1.1 (31-01-2025)

- Added `neura::Robot::getTrajectoryScalingFactor()` and `neura::Robot::getErrorCode()` to robot class.
- Added safety configuration and bridge, key switch information for cobot speed, protective stop and fullspeed mode.